

# POSTER: ActShare: Coordinating Reusable Action Requests in Multi-Agent Workflows

Rui Kang\*  
Kyoto University, Japan  
kang.rui.u25@kyoto-u.jp

Mengfei Zhu\*  
China Mobile Design Institute, China  
zhumengfei@cmdi.chinamobile.com

Tong Li<sup>†‡</sup>  
Renmin University of China, China  
tong.li@ruc.edu.cn

## Abstract

Multi-agent workflows often generate repeated action requests when specialized agents interact with shared runtime objects. When requests target the same object under explicit compatibility conditions, a single execution result can serve multiple agents. This paper presents *ActShare*, a pre-execution coordination layer for reusable action requests. Before invoking a tool or a model, each agent emits a schema-constrained structured action request. ActShare compares the request against an active action table and a recent result cache. A request is attached only to a compatible owner action; otherwise, it is executed independently. When the owner action completes, ActShare writes the result once and dispatches it to all attached requests. We prototype ActShare in a stateful multi-agent debugging workflow and evaluate effect on repeated action reduction, token and tool-call savings.

## CCS Concepts

• Computing methodologies → Multi-agent systems; Cooperation and coordination.

## Keywords

LLM agents, Reusable action requests, Action coalescing, Harness engineering

## ACM Reference Format:

Rui Kang, Mengfei Zhu, and Tong Li. 2026. POSTER: ActShare: Coordinating Reusable Action Requests in Multi-Agent Workflows. In *ACM SIGCOMM 2026 Conference (SIGCOMM Posters and Demos '26)*, August 17–21, 2026, Denver, CO, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3789240.3830289>

## 1 Introduction

Large language model (LLM)-based agents are increasingly organized as multi-agent workflows, where specialized agents cooperate through planning, retrieval, diagnosis, repair, and verification. Systems such as AutoGen, CAMEL, and MetaGPT use role-based collaboration to improve task decomposition and execution flexibility [3, 5, 10], while workflow optimization often represents agentic workflows as structured graphs or code-defined search spaces [11, 12]. However, these flexible workflows leave repeated

action requests uncoordinated, causing duplicated tool calls, rate-limit pressure, and redundant requests observed in public issue discussions [2, 4, 8].

The duplication is not merely duplicate prompting. In a debugging workflow, a failed test may become a shared runtime object when different agents ask for logs or failure context. These requests have different local purposes, so prompt caching may not match them; yet they begin with the same external work, such as reading a CI log or retrieving a trace. Executing them independently repeats LLM-backed processing, tool calls, and external I/O operations.

This paper focuses on repeated action requests that share the same target object, objective, expected output type, and related execution properties. Such requests are shareable not merely because they mention the same target or operation in text, but because they are compatible under explicit compatibility conditions. Reducing such action requests not only reduces the number of calls, but also lowers pressure on external systems and preprocessing, shortens the workflow critical path, and helps agents reason over the same state rather than independently acquired copies.

Existing work reduces redundant work through prompt/response reuse [1, 7], duplicate-call suppression with a caller-provided key [9], agent resource management [8], or serving-layer KV sharing [6]. ActShare instead coordinates agent-declared action requests over shared workflow objects. It checks declared target, state version, action, output, and side-effect fields before reusing a cached result or joining a compatible in-flight action. Thus, ActShare does not require identical prompts or histories, while requests with incomplete or incompatible declarations execute independently.

This shift makes the sharing decision harder: reusable action requests must be detected before tool or LLM execution, without relying on prompt similarity. ActShare addresses this gap by placing a coordination layer before external action execution. Each agent emits a structured action request that binds the requested action to a target object, workflow state version, output type, and side-effect class. ActShare then either reuses a valid cached result, attaches the request to a compatible in-flight owner action, or starts a new owner action. Missing, ambiguous, stale, incompatible, or state-changing requests are executed independently.

## 2 System Design

### 2.1 Architecture

ActShare operates before tool or LLM-backed action execution. Given a structured action request, it checks the workflow state and either reuses a valid recent result, attaches the request to a compatible running owner action, or starts a new owner action. ActShare maintains a recent result cache and an active action table. The owner action performs the actual execution, attached requests

\*These authors contributed equally.

<sup>†</sup>Corresponding author.

<sup>‡</sup>Also with State Key Laboratory of Internet Architecture, Tsinghua University.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM Posters and Demos '26*, Denver, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/26/08

<https://doi.org/10.1145/3789240.3830289>

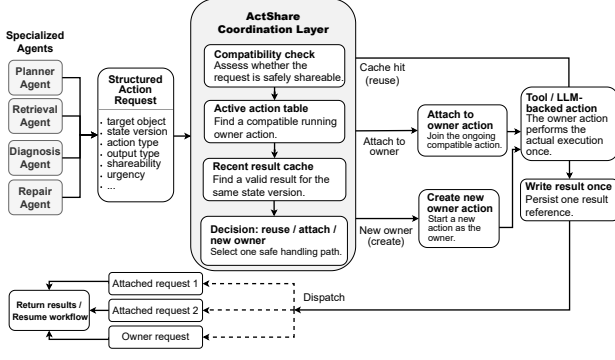


Figure 1: Architecture and request handling flow.

wait for its result, and the completed action result is written once and dispatched to the attached requests, as shown in Fig. 1.

## 2.2 Structured Action Request

A structured action request is the record that an agent submits before execution. It describes target runtime identifiers, such as request, agent, and target IDs; controlled labels, such as objective, action type, output type, shareability, and urgency; and text fields for logging and explanation. ActShare uses only runtime identifiers and controlled labels for matching. The matching path compares identifiers and labels rather than natural-language descriptions.

## 2.3 Compatibility and Runtime Control

ActShare operates only for participating agents that submit complete structured action requests conforming to a shared schema. All other actions execute independently. A request can be shared only when its declared action contract is compatible with that of an existing request. The requests must refer to the same target object and workflow state version, request the same action, and require compatible output formats and side effect properties. For each running owner action, the active action table records the owner request and attached requests. When a compatible request arrives, ActShare attaches it to the owner instead of launching another execution. After the owner completes, ActShare writes one result reference and dispatches it to all attached requests.

The policy is conservative. Cached results are reused only while both the workflow state version and the cache entry remain valid. If an owner action fails, expires, or times out, attached requests fall back to independent execution. This design trades missed sharing opportunities for safer reuse: missed sharing only loses an optimization opportunity, whereas incompatible reuse may propagate stale or mismatched results.

## 3 Evaluation

We evaluate ActShare in a compiled LangGraph workflow with planner, retrieval, diagnosis, repair, and verification agents. Each agent emits an ActionRequest before invoking a tool or an LLM-backed action. We compare *Independent*, *Prompt reuse*, *ActShare*, and *offline best sharing*, an offline baseline that sees the full request stream and merges every safely shareable action group under the same sharing rules. Fig. 2 summarizes the results.

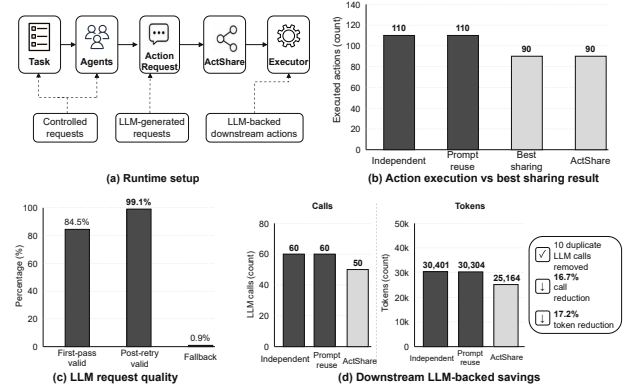


Figure 2: Evaluation in multi-agent workflow.

**Controlled validation.** With deterministic requests, independent execution performs 110 actions and offline best sharing performs 90. ActShare also performs 90 actions, showing that it captures all safely shareable actions in this workload. The required trace appears in the logs, and false reuse is zero.

**LLM-generated requests and downstream savings.** In this experiment, glm-4.5-air generates structured action requests using JSON-mode generation, Pydantic validation, and retry. Across 10 tasks, the generator reaches 84.5% first-pass schema validity and 99.1% post-retry validity. On the accepted LLM-generated request stream, independent execution performs 110 actions, offline best sharing performs 90, and ActShare also performs 90. We then replay the same accepted stream with real LLM-backed downstream actions. ActShare reduces downstream LLM calls from 60 to 50 and tokens from 30,401 to 25,164 under conservative compatibility checks. These suggest that coordinating reusable actions at the workflow layer reduces downstream model pressure without requiring agents to share prompts, histories, or model backends.

**Takeaway.** ActShare matches offline best sharing under both controlled and LLM-generated request streams. With real downstream LLM-backed summarization, it reduces model calls and tokens by removing all safely shareable duplicate downstream LLM actions in this workflow.

## 4 Conclusion

This paper presented *ActShare*, a runtime coordination layer for sharing safely reusable agent actions; in a compiled multi-agent workflow, ActShare matches offline best sharing under both controlled and LLM-generated request streams, and reduces downstream LLM-backed calls by 16.7% and downstream tokens by 17.2%. Future work will integrate ActShare into native multi-agent runtimes and evaluate it on open software-engineering tasks with real repositories, CI logs, and patch validation.

## Acknowledgments

This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFB2906701, in part by the National Natural Science Foundation of China under Grant 62572473, and in part by the State Key Laboratory of Internet Architecture under Grant HLW2025ZD17.

AI disclosure: ChatGPT was used for language polishing and Codex for evaluation-code assistance; authors reviewed all content.

## References

- [1] Sarthak Chakraborty, Suman Nath, Xuchao Zhang, Chetan Bansal, and Indranil Gupta. 2025. Generative Caching for Structurally Similar Prompts and Responses. In *Advances in Neural Information Processing Systems*.
- [2] DeerFlow GitHub Issue #1055. 2026. Agent Gets Stuck in Repetitive Tool Call Loops Until Recursion Limit. <https://github.com/bytedance/deer-flow/issues/1055>. Accessed: 2026-04-24.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*.
- [4] LangChain GitHub Issue #26019. 2024. Prevent Infinite Tool Call Loop in Customer Support Agent (LangGraph). <https://github.com/langchain-ai/langchain/issues/26019>. Accessed: 2026-04-24.
- [5] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [6] Yuhan Liu, Yuyang Huang, Jiayi Yao, Shaoting Feng, Zhuohan Gu, Kuntai Du, Hanchen Li, Yihua Cheng, Junchen Jiang, Shan Lu, Madan Musuvathi, and Esha Choukse. 2026. DroidSpeak: KV Cache Sharing Across Fine-tuned Model Variants. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association.
- [7] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2026. vCache: Verified Semantic Prompt Caching. In *International Conference on Learning Representations (ICLR)*.
- [8] Jianshu She. 2026. AgentRM: An OS-Inspired Resource Manager for LLM Agent Systems. *arXiv preprint arXiv:2603.13110* (2026). <https://arxiv.org/abs/2603.13110>
- [9] The Go Authors. 2026. singleflight: Duplicate Function Call Suppression. <https://pkg.go.dev/golang.org/x/sync/singleflight>. Package documentation for golang.org/x/sync/singleflight. Accessed: 2026-04-30.
- [10] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Language Modeling (COLM)*.
- [11] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations (ICLR)*.
- [12] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 62743–62767.